week4_notebook_student

September 23, 2026

1 HS4002 Week 4

1.1 Task for the day

We'll investigate three relationships: 1. Age and annual income 2. Income and cultural omnivorousness 3. Race and highest education level attained

For each question: (a) visualize, then (b) compute a bivariate statistic.

[50]:
```
# !pip install pandas numpy scipy plotnine

import pandas as pd
import numpy as np
import plotnine as p9
import warnings
warnings.filterwarnings('ignore')
from scipy import stats
```

1.2 Data Preparation

[51]:
```
raw_df = pd.read_csv("gss2022_mini.csv")

chosen_variables = [
        "id", "age", "yrlvmus", "yrartxbt", "yrmovie", "yrcreat",
        "income16", "prestg10", "degree", "race"
]
df = raw_df[chosen_variables].dropna().copy()

income_map = {
        1: 500,    2: 2000,   3: 3500,   4: 4500,   5: 5500,   6: 6500,
        7: 7500,   8: 9000,   9: 11250,  10: 13750, 11: 16250, 12: 18750,
        13: 21250, 14: 23750, 15: 27500, 16: 32500, 17: 37500, 18: 45000,
        19: 55000, 20: 67500, 21: 82500, 22: 100000, 23: 120000, 24: 140000,
        25: 160000, 26: 250000
}
df['income_cont'] = df['income16'].map(income_map)

# first asks if value is equal to 1 or not, then marks 1 as true and other␣
 ↪values as false
```

```
# as.type(int) then turns these true/false values into 1/0
df['binary_artxbt'] = (df['yrartxbt'] == 1).astype(int)
df['binary_movie']  = (df['yrmovie']  == 1).astype(int)
df['binary_creat']  = (df['yrcreat']  == 1).astype(int)
df['binary_lvmus'] = (df['yrlvmus'] == 1).astype(int)
df['omni'] = df[['binary_lvmus','binary_artxbt','binary_movie','binary_creat']].
 ↪sum(axis=1) # axis=1 tells python to add across the rows, not down them
df
```

[51]:

	id	age	yrlvmus	yrartxbt	yrmovie	yrcreat	income16	prestg10	\
4	5	62.0	2.0	2.0	2.0	2.0	15.0	38.0	
17	18	80.0	1.0	1.0	1.0	2.0	24.0	64.0	
18	19	35.0	1.0	2.0	1.0	2.0	9.0	48.0	
20	21	34.0	2.0	2.0	2.0	2.0	18.0	25.0	
22	23	65.0	2.0	2.0	2.0	2.0	17.0	38.0	
…	…	…	…	…	…	…	…	…	
3523	3525	29.0	1.0	2.0	1.0	2.0	21.0	32.0	
3531	3533	66.0	2.0	2.0	1.0	1.0	21.0	47.0	
3534	3536	69.0	2.0	2.0	2.0	2.0	22.0	45.0	
3535	3537	66.0	1.0	1.0	2.0	2.0	18.0	61.0	
3537	3539	73.0	2.0	2.0	2.0	2.0	9.0	33.0	

	degree	race	income_cont	binary_artxbt	binary_movie	binary_creat	\
4	1	1.0	27500	0	0	0	
17	3	1.0	140000	1	1	0	
18	0	2.0	11250	0	1	0	
20	1	3.0	45000	0	0	0	
22	3	1.0	37500	0	0	0	
…	…	…	…	…	…	…	
3523	1	1.0	82500	0	1	0	
3531	4	1.0	82500	0	1	1	
3534	1	1.0	100000	0	0	0	
3535	3	1.0	45000	1	0	0	
3537	0	1.0	11250	0	0	0	

	binary_lvmus	omni
4	0	0
17	1	3
18	1	2
20	0	0
22	0	0
…	…	…
3523	1	2
3531	0	2
3534	0	0
3535	1	2
3537	0	0

```
[709 rows x 16 columns]
```

2 Investigating the relation between age and annual income

2.1 Calculating covariance

[52]:
```
# np.cov returns a 2x2 matrix. [0,1] tells python which part of the matrix to␣
 ↪draw the covariance from
cov_val = np.cov(df['age'], df['income_cont'])[0, 1]  # calculate covariance␣
 ↪between age and income
print(f'Covariance(age, income): {cov_val:.2f}')
# interpretation: since covariance is positive, age and income tend to increase␣
 ↪together

np.cov(df['age'], df['income_cont'])
# explaining the values in the matrix:
# top left is the variance of the age, bottom right is the variance of the␣
 ↪income
# the other 2 values are the covariance of age and income!
```

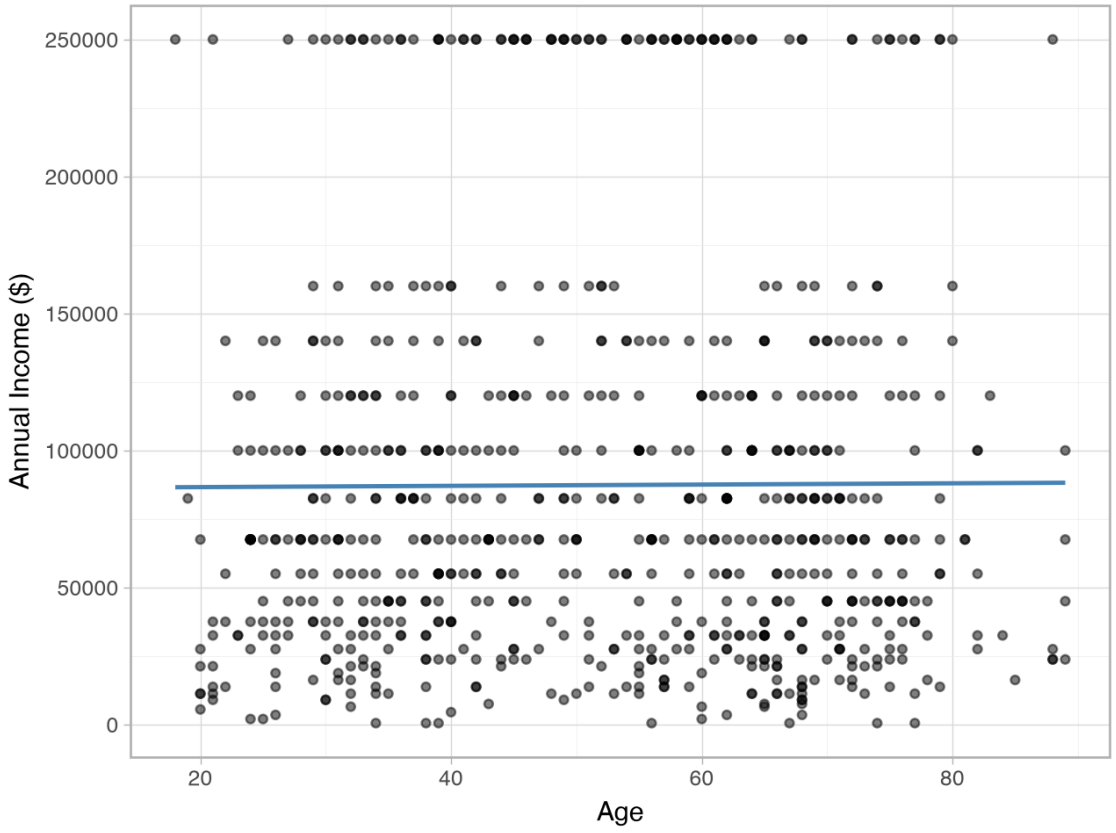
```
Covariance(age, income): 6961.46
```

[52]:
```
array([[3.01191425e+02, 6.96146150e+03],
       [6.96146150e+03, 5.17655627e+09]])
```

2.2 Data Visualisation

[53]:
```
(
        p9.ggplot(df, p9.aes(x='age', y='income_cont')) +
        p9.geom_point(alpha=0.5) + # alpha determines intensity of colour of␣
 ↪dots
        p9.geom_smooth(method='lm', se=False, color='steelblue') + # adds a␣
 ↪best fit line to the graph. se=False tells python not to add the shaded␣
 ↪confidence interval to the graph
        p9.theme_light() +
        p9.labs(x='Age', y='Annual Income ($)')
)
```

[53]:



2.3 Bivariate statistic

Pearson correlation — appropriate for two continuous variables. Comment on the magnitude, direction, and p-value.

[54]:
```
r, p = stats.pearsonr(df['age'], df['income_cont'])
print(f'Pearson r = {r:.4f}, p-value = {p:.4f}')
# Since r is very close to zero and positive, there is a very weak positive␣
↪linear correlation between age and income
# since p-value=0.8822 > 0.05, there is insufficient evidence that age has an␣
↪effect on income.
```

```
Pearson r = 0.0056, p-value = 0.8822
```

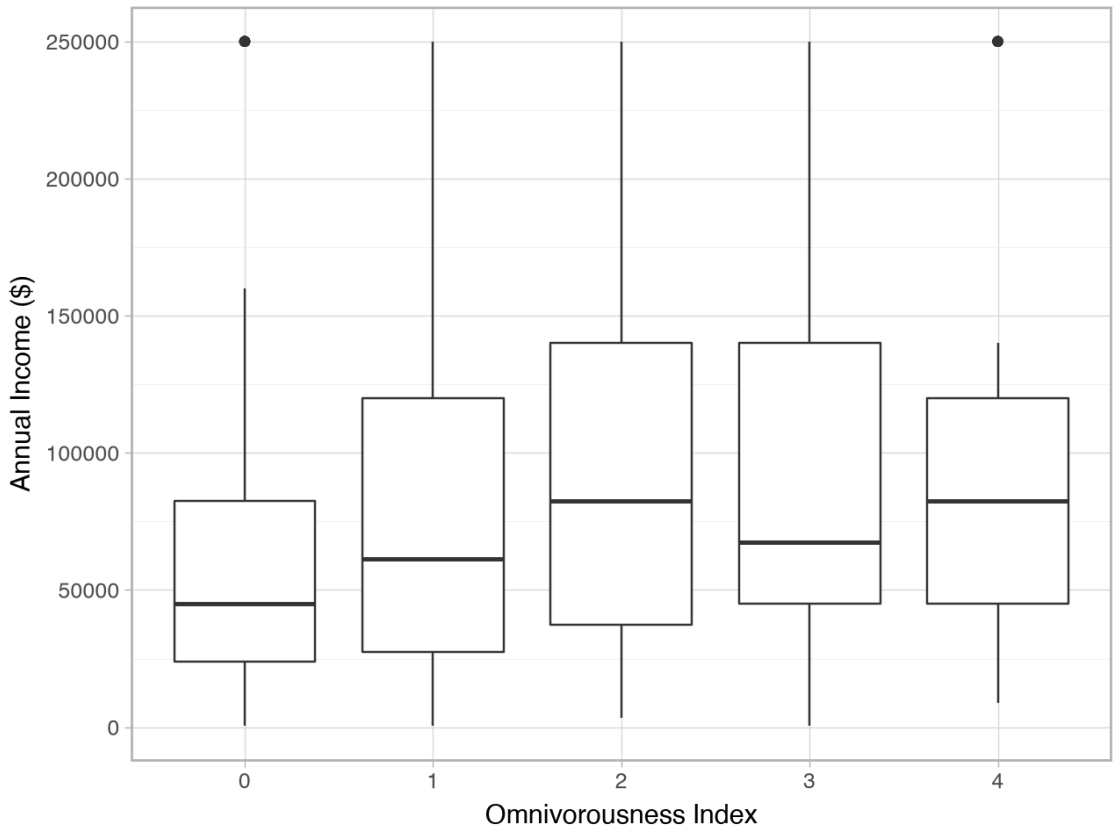
3 Investigating income and cultural omnivorousness

3.1 Data Visualisation

Omnivorousness is ordinal (0–4), so a boxplot is appropriate.

[55]:
```
(
        p9.ggplot(df, p9.aes(x='factor(omni)', y='income_cont')) +
        p9.geom_boxplot() +
        p9.theme_light() +
        p9.labs(x='Omnivorousness Index', y='Annual Income ($)')
)
```

[55]:



3.2 Bivariate statistic

Spearman correlation — appropriate when one variable is ordinal.

[56]:
```
rho, p = stats.spearmanr(df['omni'], df['income_cont'])
print(f'Spearman rho = {rho:.4f}, p-value = {p:.4f}')
# since rho = 0.2072, there is a weak monotonically increasing relationship. As␣
 ↪omnivoureness index increases, income tends to increase slightly, but this␣
 ↪relationship is weak
# Since p-value = 0, there is sufficient evidence that omnivoureness index␣
 ↪affects one's income.
```

```
Spearman rho = 0.2072, p-value = 0.0000
```

4 Investigating race and highest education level attained

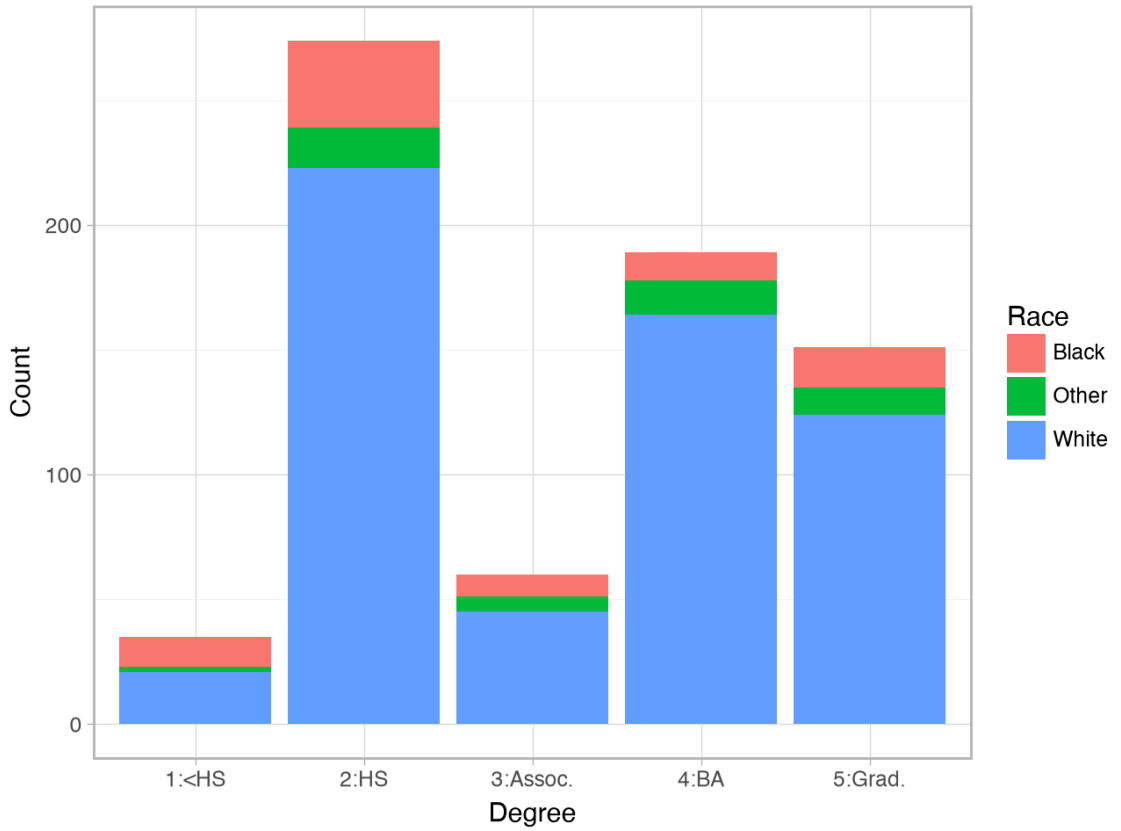
4.1 Recoding variables

[57]:
```
df['race_recoded']   = df['race'].map({1: 'White', 2: 'Black'}).fillna('Other')␣
 ↪# turn 3 = other values and NA values into 'Other'
df['degree_recoded'] = df['degree'].map({
        0: '1:<HS', 1: '2:HS', 2: '3:Assoc.', 3: '4:BA', 4: '5:Grad.'
}) # HS stands for high school
```

4.2 Preliminary visualisation

[58]:
```
(
        p9.ggplot(df) +
        p9.geom_bar(p9.aes(x='degree_recoded', fill='race_recoded')) +
        p9.theme_light() +
        p9.labs(x='Degree', y='Count', fill='Race')
)
```

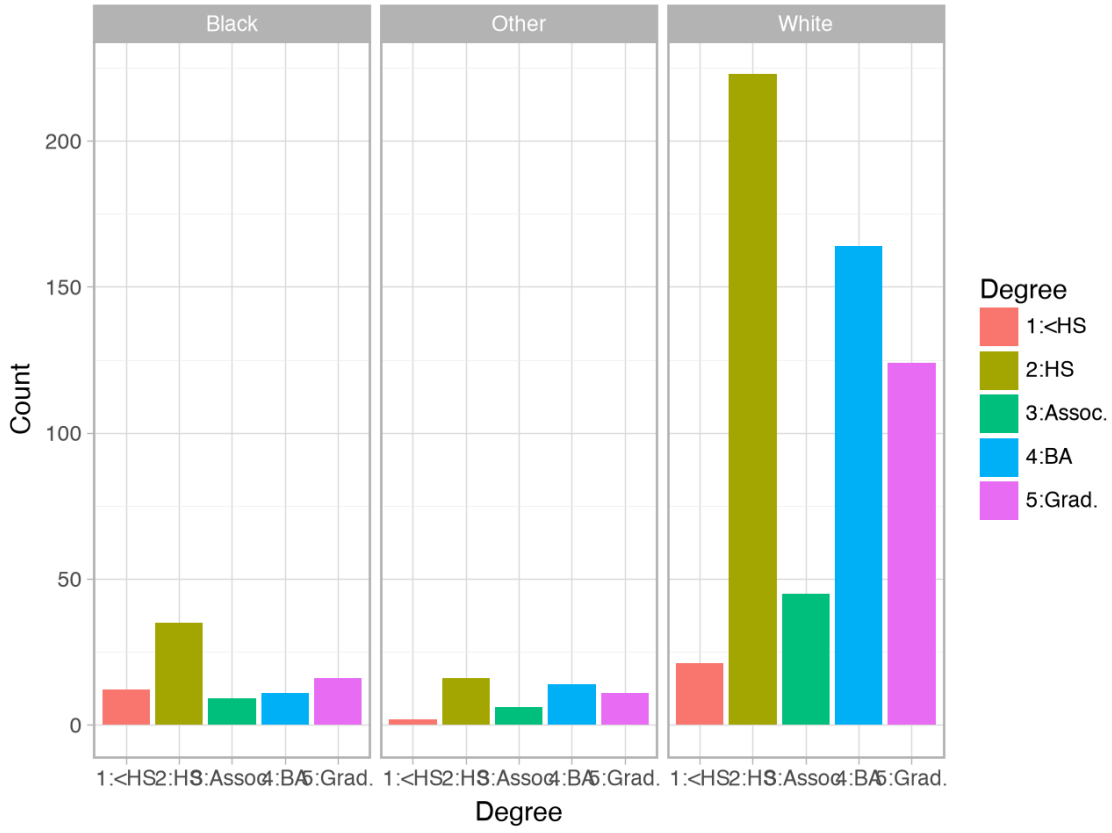
[58]:



[59]:
```
(
        p9.ggplot(df) +
```

```
        p9.geom_bar(p9.aes(x='degree_recoded', fill='degree_recoded')) +
        p9.facet_wrap('race_recoded') +
        p9.theme_light() +
        p9.labs(x='Degree', y='Count', fill='Degree')
) # easier to see relationship between race and education like this
```

[59]:



4.3 Bivariate statistic: Chi-square and Cramér's V

Chi-square tests if there is a statistically significant association. Cramér's V measures the strength of that association (0 = no association, 1 = perfect).

[60]:
```
# create a table of race by education. need to do this in order to have␣
↪chi-squared test
contingency = pd.crosstab(df['race_recoded'], df['degree_recoded'])
contingency
```

[60]:

degree_recoded	1:<HS	2:HS	3:Assoc.	4:BA	5:Grad.
race_recoded					
Black	12	35	9	11	16
Other	2	16	6	14	11
White	21	223	45	164	124

[ ]:
```
# conduct a chi squared test to determine if there is association between 2␣
 ↪categorical variables
# chi2_contingency conducts chi-squared test
chi2_result = stats.chi2_contingency(contingency)
print(chi2_result)
# chi-squared statistic is 23.223, p-value is 0.000961, degrees of freedom = 8
# expected frequency array shows the values we would expect to see in the table␣
 ↪above if race and education were truly independent (i.e. no association)
# Since p-value < 0.05, there is sufficient evidence to reject the null␣
 ↪hypothesis. race and education ar related
```

```
Chi2ContingencyResult(statistic=np.float64(26.223597706656623),
pvalue=np.float64(0.00096163894136679), dof=8, expected_freq=array([[
4.09732017,  32.07616361,   7.02397743,  22.12552891,
        17.67700987],
       [  2.41889986,  18.93653032,   4.14668547,  13.06205924,
        10.43582511],
       [ 28.48377997, 222.98730606,  48.82933709, 153.81241185,
       122.88716502]]))
```

[62]:
```
# association calculates degree of assocation between 2 nominal variables.␣
 ↪method chosen is cramer's V
cramerv_result = stats.contingency.association(contingency, method='cramer')
print(f"Cramer's V = {cramerv_result:.4f}")
# since cramer's V = 0.1360, there is a weak association between race and␣
 ↪education
```

```
Cramer's V = 0.1360
```